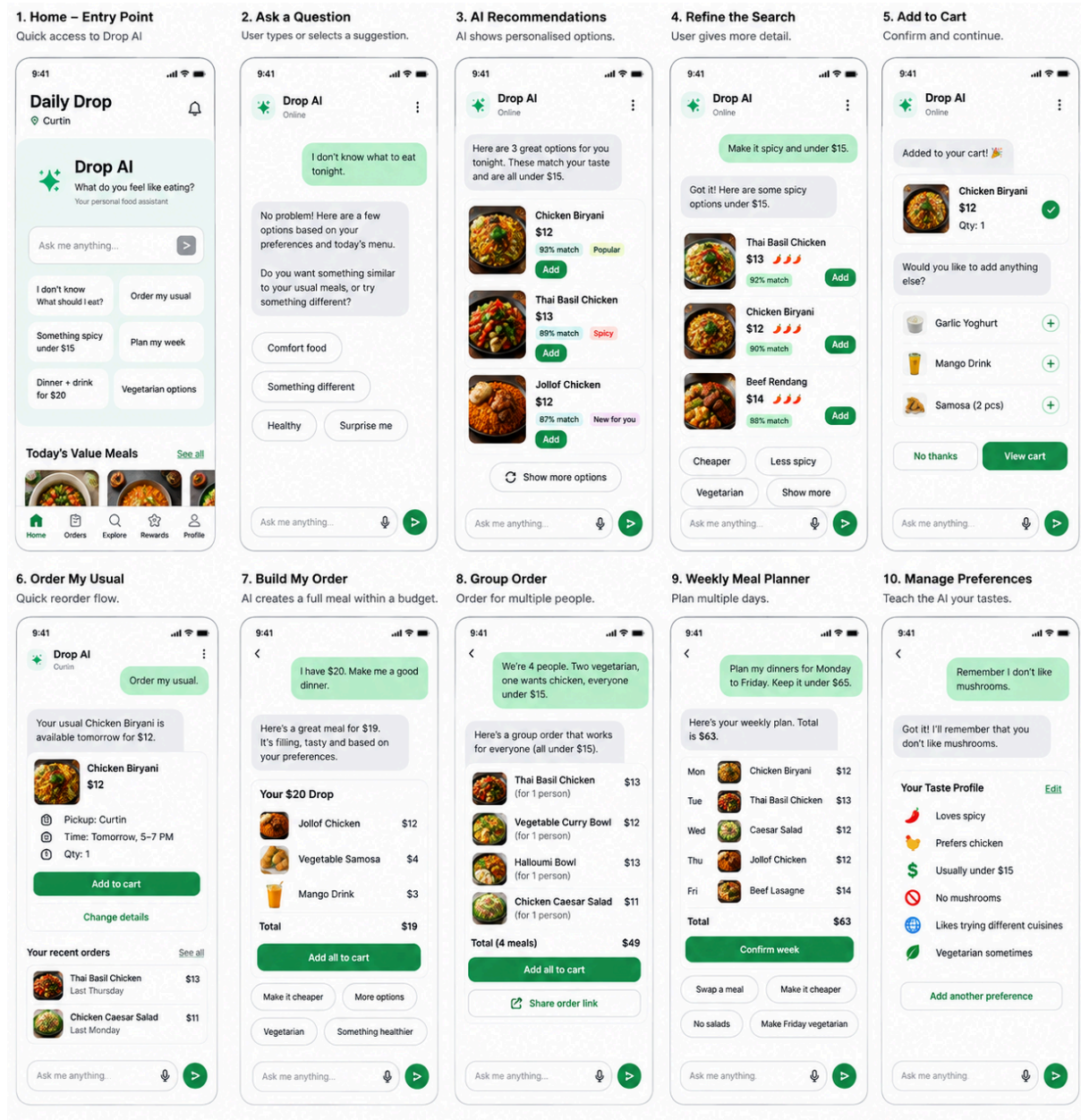


::::Section 1 - Drop AI: Your Personal Food Assistant

1. Drop AI - Your Personal Food Assistant



This could be a strong AI direction for Daily Drop because it solves a very practical problem: **people don't necessarily want to browse 30 meal cards—they want to decide what to eat quickly.**

I would think of it less as an "AI chatbot" and more as a **personal food concierge that can actually build the order.**

0. Daily Drop AI — "What should I eat?"

The entry point could simply be a prominent box on the home screen:

What do you feel like eating? ✨

Ask me anything...

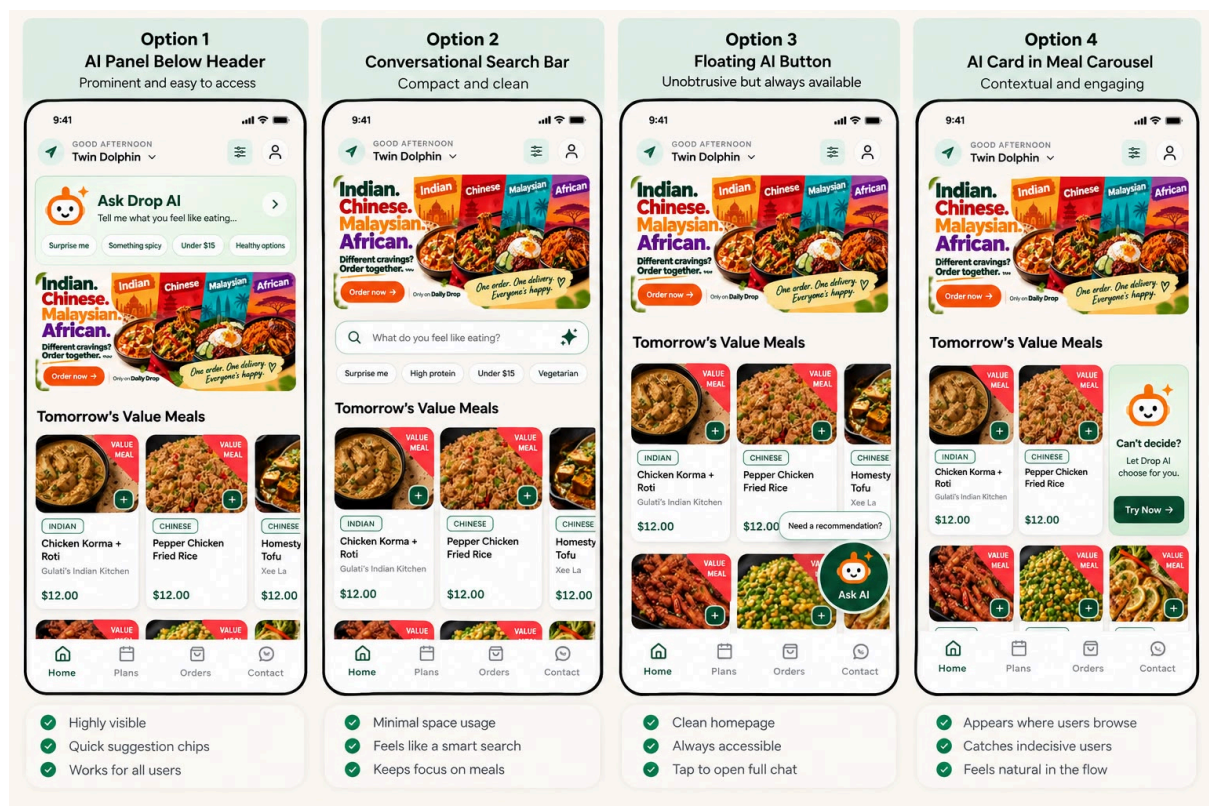
"Something spicy under \$15"

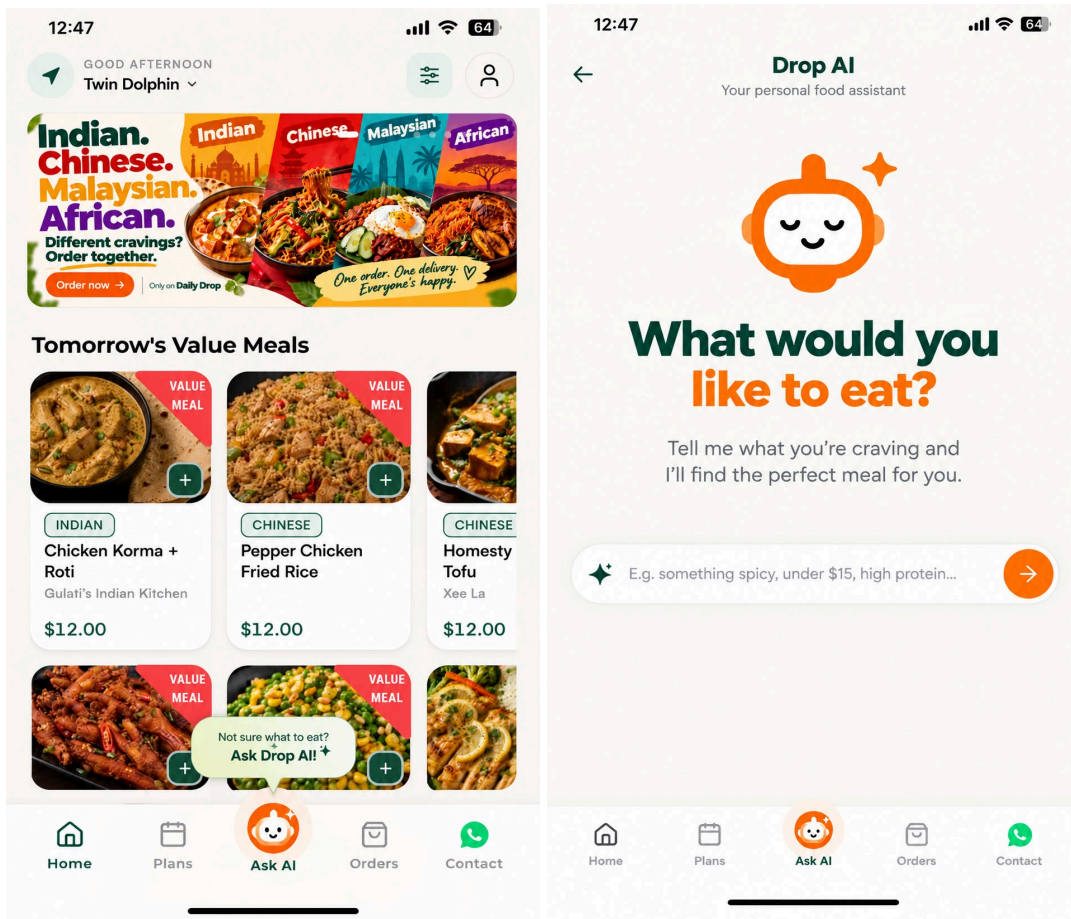
"I don't know. Pick for me."

"Healthy but filling"

"What did I have last Tuesday?"

The AI has access to **today's actual Daily Drop** menu, prices, restaurant information, pickup/delivery availability, the customer's preferences and previous interactions. So it doesn't give generic food advice. It helps them **complete an order**.





1. "I don't know what to eat"

Probably the most important use case.

Customer: **I don't know what to eat tonight.**

AI: You've been having a lot of chicken and rice lately. Want something similar or something different? **Comfort food · Something different · Healthy · Surprise me**

Customer taps: **Something different**

AI: Try the **Beef Rendang Bowl — \$13**. It's rich and mildly spicy, and people who like some of your usual meals also tend to choose it.

Add to order — \$13

Show me another

Notice that we're reducing the number of decisions.

2. Natural-language ordering

This is where AI could genuinely make Daily Drop easier than a normal ordering app.

Instead of: **Restaurant** → **Menu** → **Filters** → **Dish** → **Options** → **Cart**

the customer can simply say:

Get me a filling chicken meal under \$15 for tomorrow dinner.

AI searches tomorrow's eligible menu and presents perhaps three options.

Chicken Biryani (Indian)

Penne Pasta with Chicken (Italian)

Chicken Me Goreng (Asian)

OR:

I want Indian food but no rice.

Anything spicy and vegetarian?

Show me the cheapest chicken meals.

Something high in protein.

I have \$25. Get me dinner and a drink.

This effectively makes **conversation the search/filter interface**.

3. "Order my usual"

This could be extremely powerful for repeat customers.

Customer: **Order my usual.**

AI: **Your usual Chicken Biryani is available tomorrow for \$12.**

Chicken Biryani × 1

Curtin pickup

Tomorrow 5–7 PM

Add to cart — \$12

OR:

Get me what I had last Thursday.

Repeat last week's Wednesday order.

Get dinner for the whole week but don't repeat anything.

The customer doesn't have to find anything.

4. AI Meal Finder

Some people won't want an open-ended conversation.
Give them an extremely quick guided flow.

Help me pick dinner

What's the mood?

-  Comforting | Just like home
-  Spicy
-  Light
-  High protein
-  Cheap
-  Surprise me

Then: How hungry are you?

-  Light
-  Normal
-  Starving

Then:

I've got 3 suggestions for you

Chicken Biryani — \$12 (95% match)

Jollof Chicken — \$12 (91% match)

Thai Basil Chicken — \$14 (87% match)

This combines **AI + buttons**, which will often be much quicker than making users type everything.

5. "Build my order"

This goes beyond recommending the main dish.

Customer:

I've got \$20. Make me a good dinner.

AI could construct:

Your \$20 Drop

 **Jollof Chicken — \$12**

 **Vegetable Samosa — \$4**

 **Mango Drink — \$3**

Total \$19

Add all to cart

OR:

Dinner for 3 people under \$40.

The AI can optimise the basket around **budget + preferences + available items**.

That could be a fantastic feature for students.

6. Group ordering becomes interesting

Imagine:

We're four people. Two vegetarians, one wants chicken, everyone wants something under \$15.

AI could build the entire group order.

Or eventually: **Start a group Drop**

Send friends a link.

Everyone tells the AI what they like, and it builds the combined order.

That adds a [social component](#) to Daily Drop rather than simply copying food-delivery apps.

7. Weekly AI Ordering

This fits your pre-order model especially well.

Customer: **Sort my dinners Monday–Friday. Keep it under \$65.**

AI:

	Dinner
Mon	Chicken Biryani — \$12
Tue	Thai Basil Chicken — \$13
Wed	Caesar Chicken Salad — \$12
Thu	Jollof Chicken — \$12
Fri	Beef Lasagne — \$14

Total: \$63

Then give them: **Confirm Week · Swap meals · Make cheaper**

And they could conversationally edit it:

"Change Wednesday. Give me Butter Chicken instead."

"No salads."

"Make Friday vegetarian."

"Keep everything under \$60."

This could turn the **pre-order limitation into a product advantage**: Daily Drop becomes the app that helps you *plan food*, rather than an app you open only after you're already hungry.

8. AI should understand context

Eventually the conversation could become remarkably simple.

Customer opens Daily Drop at 7:15 PM:

What's good tomorrow?

The AI already knows:

Tomorrow's menu
Customer's pickup point
Previous orders
Likes/dislikes
Price behaviour
Dietary preferences they have chosen to provide
Today's available inventory

So:

Three good options for you tomorrow:

 **Thai Basil Chicken — \$13** (Similar to meals you've liked before.)

 **Jollof Chicken — \$12** (You haven't tried this one yet.)

 **Chicken Caesar — \$11** (Lighter option.)

Pick for me 

That is much better than presenting 50 cards.

9. Give the AI memory — but visibly

I'd let customers explicitly teach their food assistant.

Remember I don't like mushrooms.

I prefer chicken over beef.

I normally want meals under \$15.

I like spicy food.

I want to try more African food.

Then have a visible:

My Taste

-  Loves spicy
-  Prefers chicken
-  Usually under \$15
-  No mushrooms
-  Likes trying different cuisines

Edit preferences

For important dietary restrictions/allergies, I would keep explicit user-set filters **separate from inferred preferences** and **restaurant-provided allergen information** rather than having the AI guess.

10. The AI can explain recommendations

Don't just display: **93% match**

Give a short reason:

Thai Green Curry — 93% match

Spicy ✓

Chicken ✓

Under \$15 ✓

You've liked similar Thai dishes ✓

That builds trust and gives the percentage meaning.

11. Voice could make this even easier

Eventually:

 **"Hey Daily Drop, find me dinner for tomorrow."**

"Sure. Something healthy or comfort food?"

"Comfort food. Under fifteen bucks."

"I've got Chicken Biryani for \$12 and Beef Rendang for \$14."

"Biryani."

"Added. Want a drink?"

That starts feeling like an actual **ordering assistant**, rather than another food marketplace.

12. Don't make everything conversational

This is important. A pure ChatGPT-style blank screen could actually make ordering **slower**. The strongest interface is likely:

Conversation + visual meal cards + quick actions

For example:

What are you looking for?

"Something spicy under \$15"

Then AI responds with three **actual food cards with photos**:

 · **Thai Basil Chicken** \$13 · **ADD**

 · **Jollof Chicken** \$12 · **ADD**

 · **Chicken Biryani** \$12 · **ADD**

Then underneath:

Cheaper | Healthier | More options | Surprise me

So AI handles **intent**, while your existing UI handles **selection and checkout**.

13. ✨ Drop AI - Product Vision

I wouldn't call it merely a **Daily Drop chatbot**. I'd make it a core product, which is like **Your personal food assistant**

And give it four primary jobs:

FIND ⇒ "Find me something spicy under \$15."

CHOOSE ⇒ "I can't decide. Pick something for me."

BUILD ⇒ "Build me dinner + drink for \$20."

PLAN ⇒ "Sort my dinners for the week for under \$60."

Over time it learns enough that the interaction gets shorter:

New customer

"What cuisine?"

"Budget?"

"Vegetarian?"

"How hungry?"

Established customer

"Sort tomorrow."

"Chicken Biryani from your favourites is available tomorrow for \$12. Want me to add it?"

That's ultimately the AI advantage: **fewer taps between "I'm hungry" and "my dinner is sorted."**

And I'd design the system around a measurable North Star for Drop AI: **how much does it reduce time/taps from opening Daily Drop to a completed cart?**

If the normal experience takes 15–20 interactions and Drop AI gets that down to 3–5, you have a genuinely useful AI product rather than AI added for novelty.

::::Section 2 - Drop AI Dataset

2. Drop AI Dataset

If the first AI product is a **conversational personal food assistant**, I would design the database around one central objective:

Understand the customer + understand every meal + understand the current ordering context → recommend the right available meal.

You do **not** need to train a large language model from scratch. Initially, an LLM can handle the conversation, while Daily Drop's own data provides the intelligence about the customer and meals.

I'd capture data in these six groups.

1. Customer Taste Profile

This is what the assistant gradually learns about each customer.

Data	Example
Customer ID	C10293
Preferred cuisines	Thai, Indian, African
Preferred proteins	Chicken, Beef
Disliked ingredients	Mushroom
Spice preference	Hot
Vegetarian preference	Sometimes
Dietary preference	Halal
Allergies	User-declared peanut allergy
Typical budget	\$10–\$15
Portion preference	Large
Health preference	High protein
Favourite meals	Chicken Biryani
Favourite restaurants	Restaurant X
Variety preference	Adventurous
Explicit notes	"I don't like creamy food"

Importantly, distinguish between:

Explicit preference: customer actually told Daily Drop.

Inferred preference: algorithm thinks the customer likes chicken because 70% of their orders contain chicken.

I would **never** have AI silently convert an inference into a declared dietary requirement or allergy.

2. Meal Intelligence

This is probably the **most important database work to start now**.

At the moment you might have: Chicken Biryani — \$12

For AI, that isn't enough. You want something more structured:

Attribute	Example
Meal ID	M234
Name	Chicken Biryani
Restaurant	ABC Kitchen
Cuisine	Indian
Sub-cuisine	Hyderabadi
Meal category	Main
Protein	Chicken
Base	Rice
Primary ingredients	Chicken, basmati rice
Secondary ingredients	Onion, tomato, yoghurt
Spice level	Medium
Vegetarian	No
Vegan	No
Halal	Verified / Unknown

Gluten information	Restaurant supplied
Allergens	Milk
Calories	720 <i>(only if supplied/verified)</i>
Protein	38g <i>(only if supplied/verified)</i>
Portion	650ml
Price	\$12
Value/Premium	Value
Flavour profile	Spicy, savoury, aromatic
Texture	Tender, rice
Meal weight	600g
Availability	Tomorrow
Pickup locations	Curtin, Bentley
Image	URL

You don't necessarily need every field from day one. But **cuisine, protein, ingredients, dietary attributes, spice, price, portion and category** would be my minimum.

3. Every customer interaction

This is where your future recommendation model gets trained. Don't only record purchases.

Record **behaviour leading up to the purchase**.

I'd create something like an `interaction_events` table:

Customer	Meal	Action	Time
C101	M22	Impression	18:02
C101	M22	Opened	18:02
C101	M22	Added cart	18:03
C101	M22	Removed cart	18:04

C101	M35	Opened	18:05
C101	M35	Purchased	18:07

Useful events include:

meal_impression
meal_clicked
meal_detail_viewed
swiped_right
swiped_left
favourited
unfavourited
added_to_cart
removed_from_cart
purchased
reordered
rated
searched

These events have different **strengths**.

- A purchase is a **very strong positive signal**.
- A favourite is **strong**.
- A click is **weaker**.

An impression without clicking can sometimes provide useful information, but shouldn't automatically mean dislike.

4. Conversational data

Once Drop AI launches, this becomes extremely valuable.

Suppose someone says:

"I want something spicy with chicken under \$15 but I don't feel like rice."

Don't just save the conversation as text.

Save both the **raw conversation** and the **structured intent extracted** from it.

For example:

Raw message

Something spicy with chicken under \$15 but no rice

Structured representation

Field	Value
Intent	Find meal
Protein	Chicken
Spice	Spicy
Max price	\$15
Excluded ingredient/base	Rice
Meal occasion	Dinner
Date	Tomorrow

That structure becomes extremely useful later.

I'd have a `conversation_messages` table containing things such as:

`conversation_id`
`customer_id`
`timestamp`
`message`
`sender`
`intent`

And separate structured constraints/preferences.

5. Recommendation outcome data

This one is **critical and easy to forget**.

Every time AI recommends something, record:

What did we recommend and what happened afterwards?

For example:

Customer asks

"What should I eat?"

AI presents:

1. **Chicken Biryani**
2. **Jollof Chicken**
3. **Thai Basil Chicken**

The customer chooses **Jollof Chicken**.

Store:

Data	Value
Recommendation session	R9281
Customer	C101
Meal recommended	Jollof Chicken
Position	2
Recommendation reason	Similar taste
Match score	0.87
Customer clicked	Yes
Added cart	Yes
Purchased	Yes

Also record the two meals **they didn't choose**.

Over thousands of interactions the model starts learning:

Given customer X, context Y and available meals A/B/C, which meal is most likely to be selected?

That's essentially the foundation of your **recommendation system**.

6. Context

Food preference changes depending on context.

The same person might want:

Monday lunch: salad

Friday night: burger

So every recommendation/order should carry context.

Capture:

Context	Example
Date	16 Sep
Day	Wednesday
Time	7:12 PM
Meal occasion	Dinner
Order for	Tomorrow
Pickup point	Curtin
Delivery suburb	Bentley
Weather	<i>Potential later feature</i>
Group size	1
Budget stated	\$15
Ordering mode	Individual
Promotion active	None

You don't need all of these initially. Day, time, location and ordering date are particularly useful.

7. A seventh dataset I'd strongly recommend: "Why not?"

Normal ordering apps mostly collect:

What did the customer buy?

Daily Drop can deliberately collect:

Why didn't you want something?

Your swipe/game concept becomes useful here.

Customer swipes left on Beef Lasagne.

Occasionally ask:

Not for you because...

Too expensive

Don't like beef

Too heavy

Don't like pasta

Not today

That last distinction is particularly important.

"I don't like biryani"

is very different from:

"I don't want biryani today."

Your AI needs to learn the difference between **persistent preference** and **temporary intent**.

8. I'd therefore structure the core database roughly like this

You don't need one giant "AI table".

Think in connected datasets:

CUSTOMERS

customer_id
location
default_pickup_point

↓

CUSTOMER_PREFERENCES

customer_id
preference_type
preference_value

explicit/inferred
confidence
created_at
updated_at

↓

MEALS

meal_id
restaurant_id
name
description
price
portion

↓

MEAL_ATTRIBUTES

meal_id
cuisine
protein
ingredients
spice
dietary
allergens
flavour_profile

↓

MENU_AVAILABILITY

meal_id
date
location
availability

↓

INTERACTIONS

customer_id
meal_id
event_type

timestamp
session_id

↓

ORDERS

order_id
customer_id
meal_id
quantity
price
date
pickup_location

↓

AI_CONVERSATIONS

conversation_id
customer_id
message
intent
timestamp

↓

AI_RECOMMENDATIONS

recommendation_id
conversation_id
meal_id
position
reason
score

↓

RECOMMENDATION_OUTCOMES

recommendation_id
clicked
added_cart
purchased
dismissed

That gives you a very good foundation.

9. Collect from Day 1 vs later

Don't overengineer the first version.

● **Must capture immediately**

Meal data: cuisine, ingredients, protein, vegetarian/vegan/dietary flags, spice level, price, portion, restaurant and availability.

Customer behaviour: impression → click → favourite → cart → purchase → repeat purchase.

Ordering context: timestamp, date ordered for, location/pickup point and price paid.

AI interactions: question → extracted intent/constraints → meals shown → customer choice → purchase.

● **Start collecting when practical**

Swipes, explicit likes/dislikes, budget preferences, portion preference, reasons for rejection, ratings and explicit taste preferences.

● **Later**

More sophisticated taste embeddings, nutritional optimisation, contextual signals, collaborative similarity and predicted preferences.

10. AI data loop

I'd actually give this to your developers as a product requirement:

CUSTOMER SAYS

"Something spicy under \$15."

↓

AI EXTRACTS

```
spicy = yes  
max_price = 15
```

↓

SYSTEM RETRIEVES

20 eligible meals

↓

AI/RANKING SYSTEM SHOWS

3 meals

↓

CUSTOMER SELECTS

Jollof Chicken

↓

CUSTOMER PURCHASES

↓

SYSTEM RECORDS OUTCOME

↓

CUSTOMER LATER REORDERS

↓

PREFERENCE MODEL GETS STRONGER

That loop is your future AI asset.

The chatbot itself is relatively easy to reproduce. **The valuable part is the [proprietary dataset](#) connecting**

- **what customers say**
- **what Daily Drop recommends**
- **what they choose**
- **what they actually buy**
- **what they buy again.**

If you build the database correctly from the beginning, you can start with fairly simple rules/recommendations and progressively replace parts with machine-learning models as the dataset grows.

::::Section 3 - How to create the hypothetical dataset?

Yes. In fact, I would recommend creating **synthetic Daily Drop data before building the AI assistant**. It lets your developers build and test the entire recommendation pipeline without waiting months for real customers.

The key is not to create one giant Excel file with random rows. Create a **relational synthetic dataset that behaves like Daily Drop**.

1. Start with 10 connected datasets

I'd build these tables:

Dataset	Approx. synthetic size	Purpose
customers	5,000	Customer/account context
customer_preferences	20,000	Explicit/inferred taste preferences
restaurants	50	Restaurant partners
meals	500	Meal catalogue
meal_attributes	3,000+	Ingredients/taste/dietary attributes
menu_availability	30,000+	What was available where/when
interaction_events	500,000	Views, clicks, favourites, cart events, swipes
orders + order_items	50,000	Actual purchases
ai_conversations	30,000	Customer/assistant conversations

<code>ai_recommendations</code>	100,000	Recommendations and outcomes
---------------------------------	---------	------------------------------

The numbers aren't important initially. You could start with **500 customers, 100 meals and 10,000 interactions** to validate the architecture.

2. Don't make it purely random

This is where synthetic datasets often go wrong.

Suppose you generate:

Customer 001

```
prefers = chicken
spice = hot
budget = $15
```

Then their behaviour should reflect that.

If there are:

Chicken Biryani — \$12 — spicy
Thai Chicken — \$14 — spicy
Beef Lasagne — \$13 — mild
Tofu Salad — \$11 — mild

Customer 001 should have a **higher probability** of clicking/buying the first two.

Not 100%, because real humans aren't perfectly predictable.

Maybe:

Chicken Biryani → 70% likelihood
Thai Chicken → 65%
Beef Lasagne → 25%
Tofu Salad → 15%

That creates learnable patterns.

3. Build synthetic customer personas first

Instead of generating 5,000 completely random people, create perhaps **10–15 behavioural archetypes**.

For example:

Persona	Typical behaviour
Value Hunter	Usually \$10–\$13
Protein Lover	Chicken/beef, filling meals
Healthy Eater	Salads, bowls, lighter meals
Spice Lover	Strong preference for spicy dishes
Vegetarian	Vegetarian dishes
Adventurous Eater	Frequently tries new cuisines
Habitual Eater	Reorders same 3–5 dishes
Premium Foodie	Less price sensitive
Student Regular	Value meals, evening orders
Planner	Orders several days in advance
Last-Minute Buyer	Orders close to cutoff
Variety Seeker	Avoids repeating recent meals

Then give every synthetic customer a **primary persona + individual variation**.

For example:

Customer_02831

Persona: Value Hunter
Secondary: Spice Lover

Budget: \$13.50
Chicken affinity: 0.82
Beef affinity: 0.41
Vegetarian affinity: 0.20
Spice affinity: 0.91
Novelty preference: 0.56
Price sensitivity: 0.87

Now you have a hidden "ground truth" about this synthetic person.

4. Do the same with meals

Create perhaps 300 hypothetical meals.

For example:

M001 — Chicken Biryani

```
cuisine = Indian
protein = chicken
base = rice
price = 12
spice = 3/5
portion = 650ml
vegetarian = false
flavour = savoury, aromatic, spicy
health_score = 0.55
protein_score = 0.75
```

M002 — Halloumi Grain Bowl

```
cuisine = Mediterranean
protein = halloumi
price = 13
spice = 1/5
vegetarian = true
health_score = 0.82
```

And so forth.

5. Then simulate behaviour

Now you can generate a realistic sequence.

Customer opens Daily Drop.

Impression

System shows:

M001

M002

M017

M034

Your simulation calculates a hidden preference score for each meal.

Something conceptually like:

Preference Score =

Cuisine Match

- Protein Match
- Spice Match
- Budget Match
- Previous Experience
- Novelty Preference
- Context
- **Random Human Noise**

Then turn that into probabilities for:

view → **click** → **cart** → **purchase**

So your synthetic data might produce:

Customer	Meal	Event
C02831	M001	impression
C02831	M002	impression

C02831 M017 impression

C02831 M001 click

C02831 M001 add_cart

C02831 M001 purchase

That's much more useful than random event generation.

6. Then generate synthetic AI conversations

This part would be particularly useful for **Drop AI**.

You can create hundreds of conversation patterns.

For example:

Customer

Something spicy under \$15.

Store the structured intent:

```
intent = meal_search  
max_price = 15  
spice = high
```

Another:

I want chicken but not rice.

```
intent = meal_search  
protein = chicken  
exclude_base = rice
```

Another:

I'm starving but only have \$20. Get me dinner and a drink.

```
intent = build_order  
budget = 20
```

```
portion_preference = large
include = main + drink
```

Another:

Get me what I ordered last Tuesday.

```
intent = reorder
historical_reference = previous_tuesday
```

And:

Sort out my dinners for next week. Keep everything under \$65.

```
intent = weekly_plan
days = Mon-Fri
total_budget = 65
```

7. Make the conversations depend on the synthetic customer

This is important.

Don't randomly attach conversations to customers.

If **Customer 2831** is your synthetic spice-loving value hunter, they might say:

What's spicy under \$13?

and eventually buy Jollof Chicken.

A health-focused customer might say:

What's something light with lots of protein?

and choose Chicken Caesar Salad.

An adventurous eater:

Give me something I've never tried before.

This creates consistency between:

Profile → conversation → recommendation → behaviour → purchase.

8. Then simulate the AI recommendation

Suppose the customer says:

"Something spicy under \$15."

Available tomorrow:

12 qualifying meals.

Your hypothetical AI ranks them:

Rank	Meal	Synthetic score
1	Jollof Chicken	0.94
2	Thai Basil Chicken	0.91
3	Chicken Biryani	0.88

Record all three recommendations.

Then simulate the customer's response.

Maybe they click Thai Basil Chicken but eventually buy Jollof.

That's valuable training data:

recommendation → click → purchase

9. Add realistic imperfection

This is crucial if you're eventually going to test models against the data.

People don't always behave according to their preferences.

Someone who loves chicken might order vegetarian food.

Someone with a \$15 typical budget might spend \$19.

Someone might click five meals and buy nothing.

Someone might buy the same meal four weeks in a row and suddenly try something new.

Someone may say:

"Something healthy"

and ultimately buy lasagne.

That's normal human behaviour.

Your synthetic generator should therefore include **randomness/noise**, rather than producing perfectly logical customers.

10. Also simulate change over time

This makes the dataset much more sophisticated.

For example:

C1001 — January

Mostly cheap meals.

C1001 — February

Starts buying salads.

C1001 — March

Explicitly tells AI:

"I'm trying to eat more vegetarian food."

Now their behaviour shifts.

This lets you test whether your recommendation system responds to **recent behaviour instead of assuming preferences never change**.

11. I would actually create three layers of data

This will make the dataset particularly useful to your developers.

Layer 1 — Hidden synthetic truth

Your simulator knows:

C1001 chicken_affinity = 0.82

C1001 spice_affinity = 0.91

C1001 price_sensitivity = 0.87

The actual Daily Drop system **wouldn't know these numbers.**

They're only used to generate behaviour and later evaluate whether your model successfully discovers them.

Layer 2 — Observable behaviour

What Daily Drop actually sees:

view

click

swipe

cart

purchase

reorder

conversation

Layer 3 — Model inference

What your AI eventually estimates:

Chicken preference = 0.79

Spice preference = 0.88

Price sensitive = 0.81

Then you can compare:

Hidden synthetic truth vs AI prediction.

That makes the synthetic dataset useful for **testing the recommendation model**, not just testing your database.

12. I'd also deliberately create edge cases

Include synthetic customers such as:

Brand-new customer

No history whatsoever.

One-order customer

Customer with 100+ orders

Vegetarian customer

Customer with explicit allergen information

Customer who changes preferences

Customer who always orders the same meal

Customer who constantly tries new things

Customer who only responds to discounts

Customer who browses heavily but rarely buys

Customer whose favourite meal isn't available

These situations will expose problems with your AI before real customers experience them.

13. What the final dataset could look like

You could give your developers a package such as:

`daily_drop_synthetic/`

with files corresponding to the database tables:

`customers.csv`

`customer_preferences.csv`

`restaurants.csv`

`meals.csv`

`meal_attributes.csv`

`menu_availability.csv`

`interaction_events.csv`

`orders.csv`

order_items.csv
ai_conversations.csv
ai_messages.csv
ai_recommendations.csv
recommendation_outcomes.csv

Plus one special file:

synthetic_ground_truth.csv

This contains the hidden customer preferences used by the simulator.

Do not put this table into the production AI input.

It's your answer key.

14. We can actually build this now

Rather than manually inventing thousands of rows, I can generate a **proper interconnected synthetic Daily Drop dataset in Excel/CSV** with, for example:

1,000 hypothetical customers
30 restaurants
300 meals
6 months of behaviour
~100,000 interactions
~10,000 orders
AI conversations
AI recommendations and outcomes
customer preferences
meal attributes
hidden ground-truth preferences

The important part is that I can make the tables **relationally consistent**—the same customer IDs, meal IDs, restaurant IDs, order IDs and conversation IDs connect correctly across all the datasets—and make customer behaviour statistically reflect their underlying synthetic preferences.

That would give you a much more useful starting dataset for the **Daily Drop personal food assistant** than simply filling an Excel sheet with random hypothetical data.

::::Section 4 - Potential Data Collected by Daily Drop

4. Potential Data Collected by Daily Drop

Data generated	Examples	What an AI model could learn
Order history	Meal, cuisine, restaurant, sides, drink, price, quantity	Individual food preferences
Choice behaviour	Meals viewed, clicked, skipped, added, removed	Preference even when no purchase occurs
Meal attributes	Chicken, spicy, rice, high-protein, vegetarian, portion size	Relationship between meal characteristics and demand
Price behaviour	\$10 vs \$12 vs \$15 meals, discounts, bundles	Price sensitivity
Time behaviour	Order time, day of week, advance ordering	When someone is likely to order
Location	Pickup point, delivery suburb, campus	Geographic demand patterns
Restaurant behaviour	Acceptance, preparation time, availability	Supplier reliability/capacity
Fulfilment	Pickup time, delivery time, delay	Operational prediction
Ratings/feedback	Stars, repeat purchase, complaints	Meal/customer satisfaction
Search behaviour	"chicken", "halal", "healthy", etc.	Customer intent
Promotions	Notification opened → order	Which marketing actually converts
Social/game engagement	Polls, battles, quizzes, streaks	Interests beyond transactions

The really valuable dataset: the "Taste Graph"

I would deliberately build Daily Drop around capturing this.

Imagine a customer has generated these signals over several months:

Customer A

Chicken biryani → ordered 6×
Jollof chicken → ❤️

Butter chicken → ordered 4×
Beef burger → swipe left
Beetroot tofu salad → swipe left
Chicken Caesar → ordered 3×
Thai green curry → ❤️
Extra spicy → selected frequently
Meals > \$16 → rarely purchased
\$10–\$13 meals → frequently purchased
Pickup: Curtin → most weekdays
Ordering → usually 6–8 PM

Your model doesn't merely know **"this customer bought biryani."**

It begins to learn something closer to:

- prefers chicken + rice dishes
- likes medium/high spice
- adventurous across cuisines
- low interest in vegetarian meals
- moderately price-sensitive
- likes substantial portions
- orders weekday dinners
- usually orders from Curtin-area pickup points

That becomes enormously more useful.

And this connects directly to your swipe-game idea

The Tinder-style meal game you were discussing could actually become an **AI data-generation mechanism**, not merely entertainment.

For example, show:

Thai Green Curry

❤️ Swipe right

Chicken Caesar Salad

👉 Swipe left

Jollof Chicken

❤️ Swipe right

Beef Lasagne

👉 Swipe left

Butter Chicken

❤️ Swipe right

Ten swipes might take the customer **20 seconds**, but Daily Drop has suddenly collected ten explicit preference signals without requiring ten purchases.

Do this repeatedly and you can build a recommendation model much faster than relying only on orders.

You can also collect **comparative preference**, which is particularly valuable:

Biryani **VS** Butter Chicken

User picks Biryani.

Biryani **VS** Jollof Chicken

User picks Jollof.

Jollof Chicken **VS** Thai Green Curry

User picks Thai.

Now the system isn't just learning likes/dislikes. It's learning a **preference ranking**.

AI models Daily Drop could eventually build

The first model I'd pursue is a **personalised meal recommender**.

Instead of showing everybody the same 12 Value Meals, Daily Drop could show:

🔥 Your Drops Today

1. Thai Green Curry
2. Chicken Biryani
3. Jollof Chicken
4. Chicken Caesar
5. Butter Chicken

Another customer sees a completely different order.

You can eventually move beyond recommendations into several models:

Demand forecasting

Predict tomorrow's demand by meal, restaurant, suburb/pickup point and time.

"We expect 31 Chicken Biryanis around Curtin tomorrow."

That could help restaurants prepare ingredients and Daily Drop plan capacity.

Next-order prediction

Estimate when a customer is approaching another purchase and what they're likely to want.

Instead of blasting everyone:

"Order dinner!"

you could eventually trigger relevant communications around predicted intent.

Menu optimisation

If you're offering 12 Value Meals but have hundreds available from partners, AI could determine which combinations create the strongest menu.

For example:

Monday's optimal mix = 3 Indian + 2 Thai + 2 African + 2 Western + 2 salads + 1 vegetarian special.

Meal similarity model

Someone likes Jollof chicken.

Daily Drop learns that people with similar behaviour also frequently like a particular peri-peri chicken dish.

You can then surface:

Because you liked Jollof Chicken...

This becomes similar conceptually to recommendation systems used by content platforms, except the objects are **meals**.

Churn prediction

The model detects:

usually orders 3× weekly → hasn't ordered for 9 days.

Daily Drop can identify customers whose behaviour has materially changed.

Restaurant/menu intelligence

Over enough transactions, you could tell restaurant partners things like:

Your chicken dishes perform particularly well with students.

\$12–\$14 meals convert materially better than \$16+.

Spicy dishes have stronger repeat ordering.

Tuesday demand is substantially lower than Thursday.

That potentially becomes a B2B product in itself.

One thing I'd start doing now

Don't wait until you're "doing AI" to structure the data.

Every meal should have a rich set of **machine-readable attributes**, not just:

Meal = Chicken Biryani

Instead, store something more like:

Cuisine = Indian
Protein = Chicken
Base = Rice
Spice = Medium
Dietary = Halal (*where verified*)
Meal type = Dinner
Portion = 650ml
Price = \$12
Restaurant = X
Calories = X (*where reliably available*)
Protein grams = X (*where reliably available*)
Sweet/savoury = Savoury
Temperature = Hot

Then record interactions as events:

customer_123 → viewed → meal_456

customer_123 → swiped_right → meal_456

customer_123 → added_cart → meal_456

customer_123 → purchased → meal_456

customer_123 → reordered → meal_456

This event history is potentially **more valuable for future models than the raw sales table**.

And because Daily Drop is moving toward a broader lifestyle product, you could eventually build an **interest graph alongside the taste graph**—games, activities, events, offers, campus interests, etc.—provided the collection is transparent, appropriately consented, and privacy-conscious.

The long-term asset therefore isn't simply *"we have 100,000 customers."* It's **"we understand millions of preference decisions across meals, price, location, time and context."**

That is where Daily Drop could develop a genuine data advantage.

::::Section 5 - Potential Daily Drop AI Use Cases

5. Potential Daily Drop AI Use Cases

Use case	What it does	Who benefits	Potential value
Personalised Daily Menu	Ranks today's meals differently for every customer	Customer	★★★★★
"What should I eat?" AI	Gives 3–5 recommendations based on taste, budget, mood etc.	Customer	★★★★★
Taste Profile	Builds a profile such as spicy, chicken-heavy, adventurous, value-focused	Customer	★★★★★
Meal Match Score	"92% match for you" against each meal	Customer	★★★★★
Smart Meal Swiping	Learns from right/left swipes and improves future recommendations	Customer	★★★★★
Discovery Engine	Finds new meals similar customers enjoy	Customer	★★★★★
Weekly Meal Planner	Automatically proposes Mon–Fri meals	Customer	★★★★★
Smart Bundling	Predicts the side/drink most likely to accompany a meal	Customer / DD	★★★★★
Reorder Prediction	Predicts what someone may want again and when	Customer / DD	★★★★★
Demand Forecasting	Forecasts orders by dish/location/day	Restaurant / DD	★★★★★
Menu Optimisation	Determines which dishes should appear each day	DD	★★★★★
Restaurant Menu Insights	Shows restaurants what dishes/prices/customer groups perform	Restaurant	★★★★★
New Dish Prediction	Estimates likely demand before a restaurant launches a dish	Restaurant	★★★★★
Price Optimisation	Measures demand at different price points	Restaurant / DD	★★★★★
Pickup Point Forecasting	Predicts demand at each pickup location	DD	★★★★★

Delivery Planning	Forecasts volumes and helps plan routes/drivers	DD	★★★★
Churn Detection	Identifies customers whose ordering frequency is declining	DD	★★★★
Promotion Personalisation	Determines which offers are relevant to which customers	DD	★★★★
Customer Segmentation	Automatically discovers behavioural customer groups	DD	★★★★
Restaurant Matching	Identifies which new restaurants/cuisines should be added in an area	DD	★★★★★

But there are a few ideas that I think could become particularly interesting for **Daily Drop's product itself**.

1. "Drop AI" — your personal food assistant

Instead of customers navigating 50 meals:

What are you feeling like tonight?

Customer:

Something spicy, chicken, filling, under \$15.

Daily Drop already knows their historical preferences.

So it responds:

🔥 **Best match — Thai Basil Chicken**
94% match

🍛 **Chicken Biryani**
91% match

🌶️ **Jollof Chicken**
87% match

The important difference is that you're not building a generic chatbot. You're combining conversation with **Daily Drop's proprietary preference + menu + availability data**.

2. "Choose for me"

This could become a signature Daily Drop feature.

Customer presses:

 DROP FOR ME

AI considers their previous orders, swipes, dislikes, dietary preferences, price behaviour and today's menu.

Then:

We've picked tonight's Drop.

 Chicken Biryani

- Garlic yoghurt
- Mango drink

\$16.50

94% match

Accept Drop →

This turns recommendation into an actual purchasing experience.

You could even have levels:

Safe Drop → something similar to previous favourites.

Adventure Drop → something different but predicted to suit them.

Budget Drop → best match under \$12.

Healthy Drop → based on verified nutritional attributes.

3. AI Weekly Meal Planner

This fits particularly well with your **pre-order business model**.

Instead of asking customers to remember to order every evening:

Plan my week

AI generates:

Day	Your suggested Drop
Monday	Chicken Biryani
Tuesday	Thai Basil Chicken
Wednesday	Chicken Caesar Salad
Thursday	Jollof Chicken
Friday	Beef Lasagne

Week total: \$62

Customer can swipe individual dishes to replace them.

Then:

Confirm My Week

That could materially increase **orders/customer/week**.

4. AI menu construction

This is less visible to customers but potentially extremely valuable.

Suppose Daily Drop eventually has:

100 restaurants

1,500 dishes

You obviously shouldn't show 1,500 meals every day.

AI can determine which 30–50 dishes should appear tomorrow based on expected demand, variety, price, dietary requirements, location, restaurant capacity and previous performance.

For example:

Curtin — Thursday recommended menu

Chicken Biryani — predicted 42

Jollof Chicken — 31

Thai Green Curry — 27

Caesar Salad — 19

Halloumi Bowl — 17

So your marketplace itself becomes **algorithmically curated**.

5. AI demand forecasting for restaurants

Imagine the restaurant dashboard saying:

Tomorrow's forecast

Chicken Biryani

Expected: **32–39 orders**

Butter Chicken

Expected: **18–24 orders**

Paneer Rice Bowl

Expected: **8–12 orders**

Restaurant can prepare accordingly.

Because Daily Drop is predominantly **pre-order**, you have an interesting combination:

AI forecast + confirmed pre-orders.

As 9 PM approaches, uncertainty decreases.

That can potentially reduce restaurant food waste.

6. Restaurant intelligence

This could eventually become an entirely separate product.

A partner could see:

Your customers this month

Most popular: Chicken dishes

Fastest growing: Rice bowls

Highest repeat: Thai Green Curry

Best price range: \$12–\$14

Strongest day: Thursday

Weakest day: Monday

Then AI recommendations:

"Consider introducing another chicken-and-rice meal between \$12–\$14."

With enough marketplace data, Daily Drop can provide insights an individual restaurant simply cannot generate from its own customers.

7. Location intelligence

This gets particularly interesting as Daily Drop expands pickup points.

You could eventually predict:

Curtin

High demand → halal + chicken + rice

Bentley

High demand → value meals

Victoria Park

Higher demand → premium/variety

These are only hypothetical examples, but your actual data could discover patterns like these.

That helps answer:

Where should we put our next pickup point?

Which restaurants should we recruit around it?

Which meals should be available there?

What quantity should we expect?

8. AI customer segments

Rather than basic demographic segmentation such as:

18–24 / Student / Male

your AI can discover **behavioural tribes**.

For example:

The Value Hunter

Mostly \$10–12 meals.

The Protein Regular

Chicken/beef, larger portions, frequent orders.

The Explorer

Constantly tries new cuisines.

The Healthy Weekday

Salads/bowls Monday–Thursday.

The Loyalist

Orders the same 3–4 meals.

The Social Eater

Often orders multiple meals together.

These segments become useful for menu design, merchandising and communications.

9. Much bigger opportunity

Once Daily Drop becomes the **lifestyle app** you're thinking about, the recommendation engine doesn't necessarily have to stop at food.

You could have:

Your Daily Drop



EAT

92% Match — Thai Basil Chicken



DO

Student Night — nearby venue



GET

Student offer from a pickup-point partner



PLAY

Today's Food Battle

The system gradually learns:

Taste Graph

What do you like eating?

Interest Graph

What do you like doing?

Commerce Graph

What do you buy?

Location Graph

Where do you tend to engage?

Time Graph

When do you engage?

Combined, these could power a **personalised local discovery engine**.

So instead of Daily Drop ultimately being:

"an app where I order a \$12 meal"

the larger product vision could become:

"Open Daily Drop and see what's relevant to me today."

Food is potentially an excellent starting point because people make food decisions **every day**, generating repeated preference signals.

10. AI Roadmap

Phase 1 — Data foundation

Event tracking + meal tagging + customer preference profile.

Phase 2 — Recommendation

Personalised menu + swiping + meal match.

Phase 3 — Conversion

Drop For Me + weekly AI planner + bundles.

Phase 4 — Intelligence

Demand forecasting + menu optimisation + restaurant analytics.

Phase 5 — Network intelligence

Location selection + restaurant acquisition + cross-marketplace insights.

Phase 6 — Lifestyle AI

Food + offers + activities + events + games personalised into each person's **Daily Drop**.

The important strategic point is that **you don't need sophisticated AI at the beginning**. You need to make sure the product starts collecting the *right behavioural signals now*. Once you have enough interactions, progressively better models become possible.